



Rapport de Stage

Étude des Performances du Minage de Bitcoin sur
l'Architecture Processeur-in-Memory (PIM) de UpMem

Mohamed Cherif ZOUAOUI

Mai 2024

Sommaire

1	Introduction	3
2	Objectif du Stage	3
3	Organisme d'accueil	3
4	Architecture PiM de UpMem et Minage de Bitcoin	4
4.1	Présentation de l'Architecture PiM de UpMem	4
4.2	Algorithme de Minage Bitcoin	4
4.3	l'Algorithme SHA-256	5
4.4	Minage de Bitcoin sur l'Architecture PiM	7
5	Techniques d'Optimisation de l'Algorithme SHA-256 et Difficultés Rencontrées	8
5.1	Optimisation par Mise en Ligne du Code et Utilisation de Macros	8
5.2	Optimisation par Dépliage des Boucles	8
5.3	Optimisation du code Assembleur	9
5.4	Difficultés Rencontrées	12
6	Bilan d'apprentissage	13
7	Conclusion	13

1 Introduction

Ce rapport présente le travail effectué durant mon stage au Laboratoire MIS, centré sur l'optimisation de l'algorithme de hachage SHA-256 sur une architecture processeur-in-memory (PiM) développée par la société UpMem. Ce type d'architecture, intégrant 1 141 processeurs au sein de la mémoire, offre un traitement de données massivement parallèle, bien plus rapide que les architectures classiques de type Von-Neumann.

Le rapport commence par exposer l'objectif principal du stage, puis une présentation détaillée de l'architecture PiM de UpMem est fournie. Il décrit ensuite l'implémentation de l'algorithme de minage Bitcoin sur cette architecture. Les étapes suivies pour adapter et optimiser l'algorithme de hachage SHA-256 sont également détaillées, ainsi que les résultats obtenus et les difficultés rencontrées. Enfin, le bilan d'apprentissage met en lumière les connaissances et compétences appliquées et acquises.

2 Objectif du Stage

L'objectif principal de ce stage était d'étudier les performances du minage Bitcoin sur l'architecture processeur-in-memory (PiM) de UpMem et d'adapter et d'optimiser l'algorithme de hachage SHA-256 afin d'obtenir le maximum de hashrate (nombre hash par seconde (H/s)). En exploitant le microparallélisme des 1 141 processeurs intégrés dans la mémoire, la mission visait à maximiser les performances de l'algorithme, démontrant ainsi l'efficacité de cette architecture pour des tâches intensives en calcul telles que le minage de Bitcoin.

3 Organisme d'accueil

Le Laboratoire MIS (Modélisation, Information et Systèmes) est un laboratoire de recherche affilié à l'Université de Picardie Jules Verne. Il se spécialise dans plusieurs domaines de la recherche en informatique, incluant les systèmes embarqués, l'optimisation, la modélisation et la simulation, les architectures parallèles, et la gestion de l'information. Le laboratoire MIS est connu pour ses travaux novateurs et ses collaborations avec diverses industries et institutions académiques.

4 Architecture PiM de UpMem et Minage de Bitcoin

4.1 Présentation de l'Architecture PiM de UpMem

L'architecture processeur in memory (PIM) de UpMem fonctionne en intégrant des unités de traitement directement dans les blocs de mémoire. Plutôt que de transférer des données entre la mémoire et le processeur pour chaque instruction, cette approche permet de traiter les données là où elles résident, réduisant ainsi la latence et la consommation d'énergie associées aux transferts de données. Les processeurs PIM de UpMem utilisent des structures de traitement parallèle pour effectuer des opérations sur plusieurs données simultanément, exploitant ainsi pleinement le parallélisme massif disponible dans les charges de travail modernes. En résumé, l'architecture PIM de UpMem optimise l'utilisation des ressources en intégrant le traitement directement dans la mémoire, offrant ainsi des performances supérieures et une efficacité énergétique accrue par rapport aux architectures traditionnelles.

4.2 Algorithme de Minage Bitcoin

Le minage de Bitcoin est le processus par lequel de nouvelles bitcoins sont créées et les transactions sont vérifiées et ajoutées à la blockchain. Les mineurs résolvent des puzzles cryptographiques complexes pour ajouter un nouveau bloc à la blockchain. En récompense, ils reçoivent des bitcoins.

4.2.1 Préparation des données

- Les transactions sont collectées dans un bloc candidat.
- Un en-tête de bloc est créé, qui comprend des informations telles que le hachage du bloc précédent, un hachage des transactions du bloc actuel (racine Merkle), un horodatage, une version du logiciel Bitcoin, et un nonce (un nombre aléatoire).

4.2.2 Calcul du hachage

- Les mineurs prennent l'en-tête de bloc et le hachent en utilisant un double SHA-256.
- Le double SHA-256 consiste à appliquer l'algorithme SHA-256 deux fois de suite sur les données : d'abord pour obtenir un hachage intermédiaire, puis pour obtenir le hachage final.
- Le résultat final est un hachage de 256 bits (64 caractères hexadécimaux).

4.2.3 Preuve de travail

- Les mineurs ajustent le nonce dans l'en-tête du bloc et recalculent le hachage double SHA-256 jusqu'à ce qu'ils obtiennent un hachage qui commence par un certain nombre de zéros. Ce nombre est déterminé par la difficulté du réseau, qui s'ajuste périodiquement en fonction du temps nécessaire pour miner les blocs précédents.
- La difficulté est exprimée sous forme d'une cible : plus il y a de zéros au début du hachage, plus il est difficile de trouver un hachage valide.

4.2.4 Validation et ajout du bloc :

- Une fois qu'un mineur trouve un hachage qui satisfait la condition de difficulté, il diffuse le bloc validé aux autres nœuds du réseau.
- Les autres nœuds vérifient le hachage et les transactions du bloc. Si tout est correct, le bloc est ajouté à la blockchain, et le mineur reçoit la récompense (actuellement en bitcoins, plus les frais de transaction).

4.3 l'Algorithme SHA-256

L'algorithme de minage Bitcoin, appelé SHA-256 (Secure Hash Algorithm 256-bit). Voici une description de son fonctionnement :

4.3.1 Prétraitement :

- **Remplissage** : Le message d'entrée est d'abord complété pour que sa longueur soit équivalente à $448 \bmod 512$. Cela implique l'ajout d'un bit '1' à la fin du message, suivi de suffisamment de bits '0' pour atteindre cette longueur. Par exemple, si le message initial fait 448 bits, il sera complété par un '1' et 63 '0' pour atteindre 512 bits.
- **Ajout de la longueur** : Les 64 bits représentant la longueur du message original (en bits) sont ajoutés à la fin du message complété. Si le message initial fait 448 bits, ces 64 bits supplémentaires portent la longueur totale du message à 512 bits. Cela garantit que la longueur du message complété est un multiple de 512 bits.

4.3.2 Initialisation :

- Huit variables de hachage initiales (H0 à H7) sont définies. Ces variables sont des constantes spécifiques définies par le standard SHA-256 et sont initialisées comme suit : $H[8] = \{0x6a09e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a, 0x510e527f, 0x9b05688c, 0x1f83d9ab, 0x5be0cd19\}$

4.3.3 Traitement par blocs :

- Le message complété est divisé en blocs de 512 bits. Chaque bloc est traité indépendamment.
- Pour chaque bloc, un ensemble de 64 opérations est exécuté. Ces opérations impliquent des fonctions logiques et des rotations/shiftages de bits prédéfinis, ainsi que l'utilisation de constantes de ronde spécifiques (une pour chaque opération).

4.3.4 Calcul des mots de message étendus :

- Chaque bloc de 512 bits est divisé en 16 mots de 32 bits, notés W0 à W15.
- Les 48 mots de message supplémentaires, notés W16 à W63, sont générés en utilisant les mots initiaux et des opérations de mélange définies par l'algorithme. Les mots étendus sont calculés comme suit :
Pour i de 16 à 63 : $W_i = \sigma_1(W_{i-2}) + W_{i-7} + \sigma_0(W_{i-15}) + W_{i-16}$
Où σ_0 et σ_1 sont des fonctions de décalage et de rotation définies par l'algorithme.

4.3.5 Rondes de hachage :

- Pour chaque bloc, les valeurs des variables de hachage (a, b, c, d, e, f, g, h) sont mises à jour à travers 64 rondes de transformation. Chaque ronde utilise une combinaison de mots de message étendus, de variables de hachage, de fonctions logiques et de constantes de ronde (K0 à K63). Les constantes de ronde sont des valeurs spécifiques définies par le standard SHA-256.
- À chaque ronde, les valeurs des variables sont mises à jour selon les formules suivantes :

$$\begin{aligned}
 T1 &= h + \sum_1(e) + \text{Ch}(e, f, g) + Kt + Wt \\
 T2 &= \sum_0(a) + \text{Maj}(a, b, c) \\
 h &= g \\
 g &= f \\
 f &= e \\
 e &= d + T1 \\
 d &= c \\
 c &= b \\
 b &= a \\
 a &= T1 + T2
 \end{aligned}$$

Où $\sum_0$ et $\sum_1$ sont des fonctions de décalage et de rotation, Ch est une fonction de choix, et Maj est une fonction de majorité.

- **Remarque :** Dans l'implémentation, un tableau S a été utilisé pour stocker les valeurs des variables de hachage et les valeurs temporaires T1 et T2, remplaçant ainsi les variables individuelles (a, b, c, d, e, f, g, h, T1, T2) souvent utilisées dans les descriptions algorithmiques. Cette approche peut simplifier la gestion des données et rendre le code plus concis et plus lisible.

4.3.6 Mise à jour des variables de hachage :

- Après le traitement de chaque bloc, les variables de hachage initiales sont mises à jour en ajoutant leurs valeurs initiales à leurs valeurs finales après les 64 rondes de transformation. Par exemple :
- $$\begin{aligned}
 H0 &= H0 + a \\
 H1 &= H1 + b \\
 H2 &= H2 + c \\
 H3 &= H3 + d \\
 H4 &= H4 + e \\
 H5 &= H5 + f \\
 H6 &= H6 + g \\
 H7 &= H7 + h
 \end{aligned}$$

4.3.7 Production du hachage final :

- Après le traitement de tous les blocs, les variables de hachage finales (H0 à H7) sont concaténées pour produire le hachage final de 256 bits.

4.4 Minage de Bitcoin sur l'Architecture PiM

Le minage de Bitcoin sur l'architecture UpMem utilise le concept de "processeur in memory", où les opérations de calcul sont effectuées directement au niveau de la mémoire, réduisant ainsi la latence et améliorant les performances. Le processus commence par la préparation des données par le CPU, qui collecte les transactions et crée un en-tête de bloc. Cet en-tête contient le hachage du bloc précédent, la racine Merkle des transactions du bloc actuel, un horodatage, la version du logiciel Bitcoin et un champ nonce (qui sera ajusté par les DPUs). Le CPU concatène toutes les parties de l'en-tête de bloc en une chaîne de caractères, à l'exception du nonce, et envoie cette chaîne de caractères fixe aux DPUs. Chaque DPU est responsable d'une plage spécifique de valeurs de nonce. Chaque DPU concatène sa valeur de nonce spécifique à la chaîne de caractères fixe de l'en-tête de bloc, puis exécute l'algorithme de double SHA-256 sur la chaîne de caractères résultante. Le DPU vérifie si le hachage final commence par un nombre de zéros requis par la difficulté du réseau. Si le hachage est valide (c'est-à-dire qu'il répond à la difficulté), le DPU doit attendre que tous les autres DPUs terminent avant d'envoyer le nonce valide (golden nonce) au CPU. Le CPU collecte les résultats des DPUs. Si un DPU a trouvé un nonce valide, ce nonce est utilisé pour créer un bloc valide. Ensuite, le CPU diffuse le bloc validé aux autres nœuds du réseau Bitcoin, et le mineur reçoit la récompense en bitcoins, incluant les nouveaux bitcoins créés et les frais de transaction inclus dans le bloc. En traitant les données directement dans la mémoire et en utilisant des multiples DPUs pour un traitement parallèle massif, l'architecture PIM optimise le processus de minage, contribuant à une validation rapide et sécurisée des blocs dans la blockchain Bitcoin.

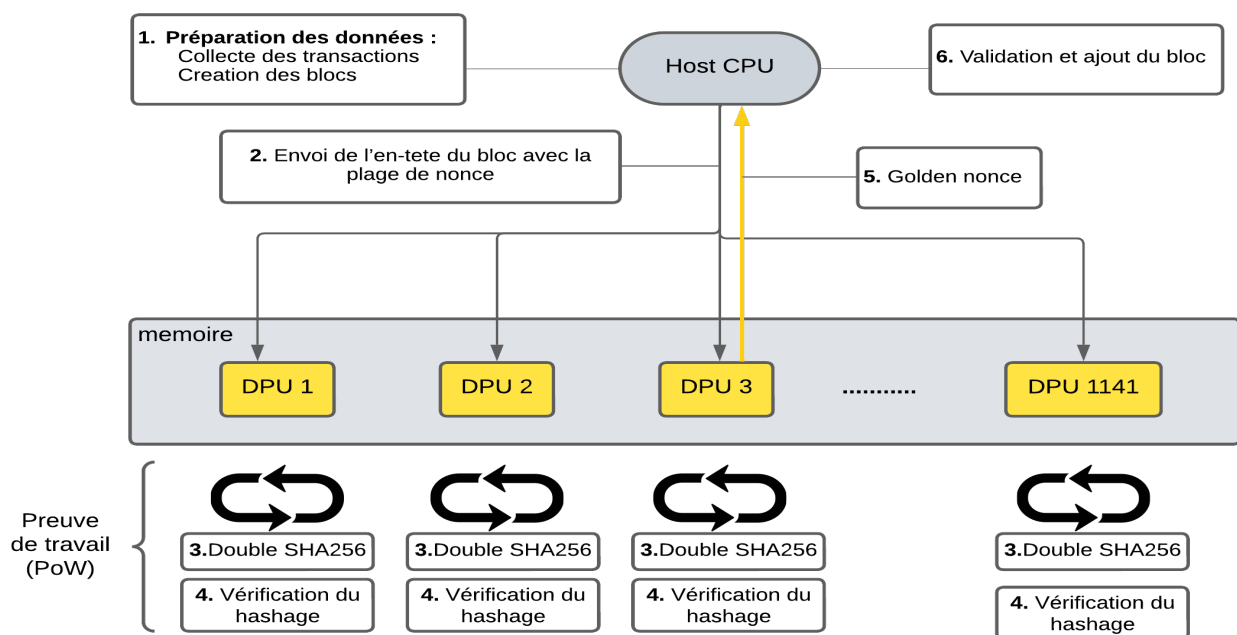


FIGURE 1 – Minage de Bitcoin sur l'Architecture PiM

5 Techniques d'Optimisation de l'Algorithme SHA-256 et Difficultés Rencontrées

Dans le cadre de l'optimisation de l'algorithme SHA-256 pour le minage de Bitcoin sur l'architecture UpMem, la première étape a consisté à coder l'algorithme SHA-256. La version initiale de cet algorithme était très basique et peu performante, ne permettant de calculer que 3 millions de hachages par seconde (3M hash/s). Cette performance était insuffisante pour le minage efficace de Bitcoin, compte tenu de la nécessité de traiter de très grandes quantités de données en peu de temps.

5.1 Optimisation par Mise en Ligne du Code et Utilisation de Macros

La première technique d'optimisation que j'ai utilisée a été la mise en ligne du code. Cela consistait à minimiser le nombre de fonctions utilisées pour rendre le code plus efficace. J'ai fini par n'utiliser que deux fonctions principales : la fonction principale sha256, qui reçoit l'en-tête de bloc et le divise en blocs de 512 bits, et la fonction consume_chunk, qui traite ces blocs et effectue le calcul du hachage. En réduisant le nombre de fonctions, j'ai réduit les appels de fonction coûteux, ce qui a amélioré les performances globales.

En outre, j'ai utilisé des macros pour optimiser davantage le code. Les macros permettent de remplacer des segments de code répétitifs par des blocs de code en ligne, éliminant ainsi le coût des appels de fonction supplémentaires et accélérant l'exécution. Cette optimisation a permis de passer de 3M hash/s à **5M hash/s**. Bien que cette amélioration soit significative, elle restait insuffisante pour un minage de Bitcoin efficace, nécessitant des performances encore plus élevées.

5.2 Optimisation par Dépliage des Boucles

Après la mise en ligne du code et l'utilisation de macros, la deuxième technique d'optimisation que j'ai utilisée est le dépliage des boucles (loop unrolling). Cette méthode consiste à réduire le nombre d'itérations d'une boucle en exécutant plusieurs itérations en une seule, ce qui permet de diminuer le coût des opérations de boucle et d'améliorer les performances globales.

Dans le cas de l'algorithme SHA-256, une boucle typique itère 64 fois, appelant une macro à chaque itération pour effectuer une partie du calcul. Avec le dépliage des boucles, au lieu de boucler 64 fois et d'appeler la macro à chaque itération, j'appelle la macro plusieurs fois dans la même itération, puis j'incrmente l'index i par le nombre d'appels effectués.

Par exemple, au lieu de :

```
for (int i = 0; i < 64; i++) {
    MACRO(i);
}
```

Le code est modifié pour :

```
for (int i = 0; i < 64; i += 4) {
    MACRO(i);
    MACRO(i + 1);
    MACRO(i + 2);
    MACRO(i + 3);
}
```

}

Cette optimisation réduit le nombre total d'itérations de la boucle, diminuant ainsi le coût associé aux contrôles et aux incrémentations de la boucle. En conséquence, le dépliage des boucles réduit le temps d'exécution des calculs répétitifs dans l'algorithme SHA-256.

Grâce à cette technique, j'ai pu augmenter encore la performance de l'algorithme. En combinant la mise en ligne du code et l'utilisation de macros avec le dépliage des boucles, la performance est passée de 5M hash/s à **9M hash/s**.

5.3 Optimisation du code Assembleur

Après avoir atteint les performances maximales possibles avec les optimisations en C, j'ai décidé de passer à un niveau inférieur en utilisant l'assembleur. L'objectif était de minimiser autant que possible les accès à la mémoire, ce qui est crucial pour améliorer davantage les performances de l'algorithme SHA-256.

Le premier défi consistait à comprendre le code assembleur généré par le compilateur. Ce code était extrêmement volumineux, avec plus de 3500 lignes d'instructions. Pour réussir cette étape, j'ai dû analyser minutieusement chaque section du code pour identifier les points où les optimisations pourraient être appliquées.

5.3.1 Étapes de l'optimisation en assembleur

1. Analyse du code généré :

- J'ai commencé par examiner le code assembleur généré par le compilateur, en identifiant les instructions clés et en repérant les sections du code où les accès à la mémoire étaient fréquents.
- Cette analyse a permis de repérer les points de redondance et les instructions inutiles qui pouvaient être éliminées ou réécrites pour être plus efficaces.

2. Minimisation des accès mémoire :

- Le principal objectif était de réduire le nombre d'accès à la mémoire. Cela impliquait de conserver autant de données que possible dans les registres du processeur, réduisant ainsi le besoin de lire et d'écrire fréquemment dans la mémoire.
- J'ai réécrit les sections critiques du code pour utiliser les registres de manière plus efficace, en utilisant le maximum de registres pour sauvegarder les données et travailler directement avec les registres, évitant ainsi d'écrire les données dans la mémoire et de les relire à chaque fois. Dans l'architecture UpMem, chaque DPU dispose de 23 registres.
- En s'assurant que les valeurs intermédiaires nécessaires pour le calcul du hachage restaient dans les registres autant que possible, j'ai pu optimiser le code de manière significative.
- Voici un exemple concret de cette optimisation :

Code C

```

#define SHA256_ROUND(S, w, k) \
do { \
    S[8] = S[7] + SIGMA1(S[4]) + CH(S[4], S[5], S[6]) + k + w; \
    S[9] = SIGMA0(S[0]) + MAJ(S[0], S[1], S[2]); \
    S[7] = S[6]; S[6] = S[5]; S[5] = S[4]; \
    S[4] = S[3] + S[8]; S[3] = S[2]; S[2] = S[1]; S[1] = S[0]; \
    S[0] = S[8] + S[9]; \
} while (0)

for (size_t i = 0; i <= 32; i += 32)
{
    SHA256_ROUND(S, w[i], k[i]);
    SHA256_ROUND(S, w[i + 1], k[i + 1]);
    SHA256_ROUND(S, w[i + 2], k[i + 2]);
    //32 itérations...
}

```

Code Assembleur généré
par le compilateur

```

lw r0, r22, -36 //lire S[7]
lw r2, r22, -48 //lire S[4]
//calculs...
lw r1, r22, -44 //lire S[5]
and r1, r2, r1
lw r3, r22, -40 //lire S[6]
//calculs...
sw r22, -32, r0 //ecrire dans S[8]

lw r4, r22, -64 //lire S[0]
//calculs...
lw r2, r22, -60 //lire S[1]
and r1, r4, r2
lw r3, r22, -56 //lire S[2]
//calculs...
sw r22, -28, r0 //ecrire dans S[9]

lw r0, r22, -40 //lire S[6]
sw r22, -36, r0 //ecrire S[6]
lw r0, r22, -44 //lire S[5]
sw r22, -40, r0 //ecrire S[5]
lw r0, r22, -48 //lire S[4]
sw r22, -44, r0 //ecrire S[4]
lw r0, r22, -52 //lire S[3]
lw r1, r22, -32 //lire S[8]
add r0, r0, r1
sw r22, -48, r0 //ecrire S[3] + S[8]
lw r0, r22, -56 //lire S[2]
sw r22, -52, r0 //ecrire S[2]
lw r0, r22, -60 //lire S[1]
sw r22, -56, r0 //ecrire S[1]
lw r0, r22, -64 //lire S[0]
sw r22, -60, r0 //ecrire S[0]
lw r0, r22, -32 //lire S[8]
lw r1, r22, -28 //lire S[9]
add r0, r0, r1
sw r22, -64, r0 //ecrire S[8] + S[9]

```

Le code généré par le compilateur lit et écrit les valeurs dans la mémoire à chaque itération, ce qui est coûteux en termes de performances. Pour pallier cela, j'ai stocké les valeurs initiales de S dans des registres (r0 à r9), permettant ainsi de manipuler ces valeurs sans accéder à la mémoire, ce qui améliore l'efficacité de l'algorithme. Cette optimisation a permis d'éliminer environ 25 accès mémoire par itération, ce qui équivaut à un total de 1600 accès mémoire éliminés par bloc. Puisque le minage de Bitcoin implique généralement deux blocs, cela signifie que j'ai éliminé 3200 accès mémoire par un seul appel SHA256, augmentant considérablement les performances de l'algorithme dans ce contexte.

Grâce à cette optimisation, la vitesse de hachage est passée de 9M hash/s à **13M hash/s**.

5.3.2 Autres idées d'optimisation

1. Une autre idée d'optimisation que j'ai explorée dans le code assembleur concerne la réduction des accès mémoire redondants en utilisant une sorte de mécanisme de cache. Considérons le code suivant :

```

#define COMPUTEW(w, i)
do {
    w[i] = SIGMA3(w[i - 2]) + w[i - 7] + SIGMA2(w[i - 15]) + w[i - 16];
    w[i + 1] = SIGMA3(w[i - 1]) + w[i - 6] + SIGMA2(w[i - 14]) + w[i - 15];
    w[i + 2] = SIGMA3(w[i]) + w[i - 5] + SIGMA2(w[i - 13]) + w[i - 14];
    w[i + 3] = SIGMA3(w[i + 1]) + w[i - 4] + SIGMA2(w[i - 12]) + w[i - 13];
    w[i + 4] = SIGMA3(w[i + 2]) + w[i - 3] + SIGMA2(w[i - 11]) + w[i - 12];
    w[i + 5] = SIGMA3(w[i + 3]) + w[i - 2] + SIGMA2(w[i - 10]) + w[i - 11];
    w[i + 6] = SIGMA3(w[i + 4]) + w[i - 1] + SIGMA2(w[i - 9]) + w[i - 10];
    w[i + 7] = SIGMA3(w[i + 5]) + w[i] + SIGMA2(w[i - 8]) + w[i - 9];
} while(0)

.....
for(int i = 16; i < 64; i += 16){
    COMPUTEW(w, i);
    COMPUTEW(w, i + 8);
}

```

Dans ce code, certaines valeurs telles que $w[i - 15]$ et $w[i - 14]$ sont utilisées deux fois par itération de la macro COMPUTEW. En conséquence, le compilateur accède à ces valeurs deux fois depuis la mémoire, ce qui entraîne des accès mémoire redondants et inefficaces.

Pour optimiser cela, j'ai implémenté une solution similaire à un cache. L'idée est de charger chaque valeur nécessaire depuis la mémoire une seule fois et de la stocker dans un registre pour une utilisation future. En faisant cela, je réduis le nombre d'accès mémoire de moitié. Cependant, malgré l'apparente efficacité de cette solution, l'amélioration des performances n'a pas été aussi significative que prévu. La performance est passée de 13M hash/s à seulement **13.4M hash/s**. Malgré tout, cette optimisation a démontré que même de petites réductions des accès mémoire peuvent contribuer à des améliorations globales des performances, même si ces améliorations sont modestes.

2. Une autre idée d'optimisation sur laquelle j'ai travaillé concerne l'accès aux constantes K_i dans la macro SHA256_ROUND. Actuellement, à chaque itération, nous devons accéder à la mémoire pour obtenir la valeur de K_i ce qui est coûteux en termes de performances. Pour résoudre ce problème, j'ai proposé d'utiliser les valeurs immédiates de K_i étant donné que nous les connaissons dès le début de l'exécution. Par exemple, au lieu d'accéder à la mémoire pour obtenir la valeur de K_0 , nous pouvons l'ajouter directement comme une valeur immédiate :
add r0, r0, **0x428a2f98**

Cette approche nous permettrait d'éliminer environ 130 accès mémoire par appel à la fonction SHA256. Cependant, cette solution nécessite le dépliage de la boucle, car si nous remplaçons directement les valeurs avec les constantes immédiates, nous ne pourrions pas boucler sans répéter les mêmes valeurs deux fois. Malheureusement, il n'est pas possible de déplier la boucle davantage, car le code deviendrait trop long et la mémoire disponible pour stocker le code (l'IRAM) serait insuffisante, ce qui entraînerait une erreur d'exécution.

5.4 Difficultés Rencontrées

- **Compréhension du code assembleur généré** : Analyser le code assembleur généré par le compilateur a été une tâche complexe en raison de sa longueur et de sa complexité. Comprendre chaque section du code était essentiel pour repérer les opportunités d'optimisation.
- **Gestion des contraintes matérielles** : L'architecture UpMem présentait des contraintes matérielles spécifiques, telles que la limitation de la mémoire disponible (IRAM) pour stocker le code. Cette contrainte a limité la portée des optimisations possibles et a nécessité des compromis dans la conception de l'algorithme.
- **Optimisation du code assembleur** : Optimiser le code assembleur pour minimiser les accès mémoire et maximiser l'utilisation des registres du processeur était une tâche délicate. Bien que certaines optimisations aient conduit à des améliorations significatives des performances, d'autres approches n'ont pas abouti aux gains escomptés ou ont même introduit des erreurs d'exécution.
- **Gestion de la complexité** : L'algorithme SHA-256 lui-même est complexe, avec de nombreuses dépendances et interactions entre les étapes de calcul. Concevoir des optimisations qui ne compromettent pas l'intégrité de l'algorithme et qui maintiennent la précision des résultats a été un défi constant.
- **Manque de ressources en ligne** : L'absence de ressources en ligne adéquates pour apprendre et résoudre les problèmes rencontrés a constitué un défi majeur, nécessitant une approche plus autonome et une exploration approfondie des solutions potentielles.

6 Bilan d'apprentissage

Pendant ce stage, j'ai acquis et développé un ensemble de connaissances et de compétences importantes, notamment :

- **Résolution de problèmes complexes** : En travaillant sur l'optimisation de l'algorithme de hachage, j'ai développé ma capacité à résoudre des problèmes complexes, à analyser les performances du code et à proposer des solutions innovantes pour améliorer l'efficacité du processus de minage.
- **Analyse approfondie du code assembleur** : J'ai appris à décortiquer et à comprendre un code assembleur complexe, ce qui m'a permis d'identifier les points d'optimisation potentiels et d'améliorer les performances de l'algorithme SHA-256.
- **Programmation en langage C** : J'ai consolidé mes compétences en programmation en travaillant sur l'optimisation de l'algorithme SHA-256 en langage C
- **Utilisation de l'architecture PIM** : J'ai appris à travailler avec l'architecture PIM, en comprenant son fonctionnement et en exploitant ses fonctionnalités pour améliorer les performances de l'algorithme de hachage.
- **Connaissance en Blockchain et Cryptomonnaies** : Pendant ce stage, j'ai eu l'opportunité d'approfondir mes connaissances dans le domaine de la blockchain, en particulier en ce qui concerne le Bitcoin. J'ai appris les principes fondamentaux de la technologie blockchain, son fonctionnement, ainsi que les applications pratiques des cryptomonnaies dans des domaines tels que le minage de Bitcoin. Cette expertise nouvellement acquise constitue un atout précieux dans un secteur en pleine expansion et en évolution constante.
- **Compétences en Cryptographie** : Renforcées par une étude approfondie de l'algorithme SHA256, j'ai acquis une solide compréhension des principes cryptographiques fondamentaux, essentiels pour assurer la sécurité et l'intégrité des données dans les transactions blockchain.

7 Conclusion

ce stage a été une opportunité précieuse pour acquérir et perfectionner diverses compétences, notamment en programmation bas niveau, en optimisation de code et en traitement parallèle. J'ai également pu explorer des domaines émergents tels que la blockchain et la cryptomonnaie, enrichissant ainsi mon bagage professionnel. Cette expérience m'a permis de relever des défis techniques complexes, renforçant ainsi ma capacité à analyser les performances et à résoudre des problèmes de manière efficace. Je suis reconnaissant de cette expérience qui m'a permis d'élargir mes compétences et mes perspectives professionnelles.